

Software Requirements Specification Document

For: *Advising Assistant - Front End*

Author(s): *Oluwajomiloju Adejumo, Keishon Boose, Rabindra Giri, and Evans Smith.
University of Southern Mississippi, Front End – Advising Assistant Group 3*

Purpose

This document outlines the functional and non-functional requirements for the Advising Assistant – Front End software application. It serves as a blueprint for the development, testing, and maintenance of the system.

Scope

The advising assistant app should be an app that can assist students with making and planning schedules for their future semester. Some other functions will be telling when the person can or cannot graduate. Also, they should be able to help students plan their schedule for the upcoming semester. The app should be able to store a curriculum that a student is enrolled in and compare the classes that have already been taken to the ones that need to be taken. And it should generate a list of classes that the student should take. This app targets college students and/or parents of future college students.

Overview

The advising assistant uses a client-server architecture to facilitate the user to navigate through their courses and allow them to perform different functions. The front end of the advising assistant will provide user with abilities to add the courses in the web application and help in tracking the courses with a checklist of what has been completed and what is remaining to calculate and show in the front end to show how much time is remaining for the completion of the degree and their track of when they are going to finish in an interactive and intuitive way. We will add an interactive user interface for students to input their academic history and add the most common courses available at the university as recommendations and visualize the progress towards graduation. The server-side architecture will handle the connection to database which will typically be done by a backend tech stack like Django or Express.js. The database will be connected through a connection pool depending on the Tech stack. As our team is designing the front end, we will be using React.js for the front end and Tailwind CSS for design. We will also be using Git as a version control system for our frontend. The primary way of sending user data will be with controls like form fields and buttons triggering RESTful APIs with axios package on the front end to the application the backend team is using, and the data will be processed in the backend and recommendation will be given for courses on basis of that.

Overall Description

The primary goals and objective of the Advising Assistant is to help students plan their semester efficiently to ensure that students stay on track of graduation while also recommending electives based on the students' interest. The system should also effectively show the accurate description of courses and helpful metrics like how many courses are left and the estimated graduation date so that advising can be simpler and more intuitive while replacing the traditional ways of complex advising and taking courses.

The users being targeted by Advising Assistant are undergraduate students who must face the complex process of advising. We realized that students must have access to a user friendly and intuitive way of tracking courses and receiving appropriate feedback on the progress of their degree. So, Advising Assistant targets these university students and brings a new experience to advising. We also understand the limitations of a computer program and have advisor's account to answer questions asked by students with a specific major.

Our target operating system will be desktop and if we are able to make the website responsible in the limited time constraints then mobile as well. We are creating a web app so the software required to interact with the advising assistant will be a browser. The app will require an internet connection to fetch the data of courses on the database from the backend to display to the users.

Functional Requirements

F1: Requirement 1 Courses Taken

Description: *The system will display past courses that users have taken in their academic progress report.*

Trigger: *Activated when the user logs in and navigates to the “Courses Taken” section.*

Inputs: *User academic history and completed courses.*

Outputs: *show all courses that the user has enrolled in the past semesters which includes Course Name, Date of Registration, the semester taken, and their final grade in that course they made while in those said course*

F2: Requirement 2 Display Recommended Courses

Description: *The system will display a list of recommended courses based on the user’s academic progress and program requirements. (Evans Smith)*

Trigger: *Activated when the user logs in and navigates to the “Recommended Courses” section.*

Inputs: *Users academic history, Major and completed courses, Course availability data*

Outputs: *A list of recommended courses, including Course name, Course code, Available sections, Registration status.*

F3: Requirement 3 Student Account Management

Description: *The system will allow a student to create an account log in, and view/update their profile and major. It shows the student's university, name, email address and other details that can be added.*

Trigger: *Activated when user clicks register or logs in and navigates to the user’s profile page.*

Inputs: *Users name, university name, university ID, email and optional details.*

Outputs: *A student account registered with necessary information. Also, a profile page that displays student information and course progress.*

Non-Functional Requirements

NF1: Intuitive and User-friendly UI for Usability

Type: *Usability*

Description: The system should have an intuitive and user-friendly interface so that a student without prior experience of using the application can register and create an account, login and use the basic functionalities without the need for any assistance. Using different user design principal architecture to fulfill these basic requirements.

NF2: Secure User Authentication

Type: *Security*

Description: *The system shall require secure user authentication to access course recommendations. User login credentials must be encrypted, and session management must prevent unauthorized access. Multi-factor authentication (MFA) should be implemented for added security. (Evans Smith)*

NF3: Modular Components and User Testing

Type: *Maintenance*

Description: The components designed should be modular and reusable so that the testing can be done without any additional time commitment and code can be reused to decrease the time commitment involved in the project. Also, timely maintenance of the code should be done, and user testing must be taken seriously before the release of the webapp like testing every feature works before moving on.

NF4: Stable Performance and Optimized Loading Time

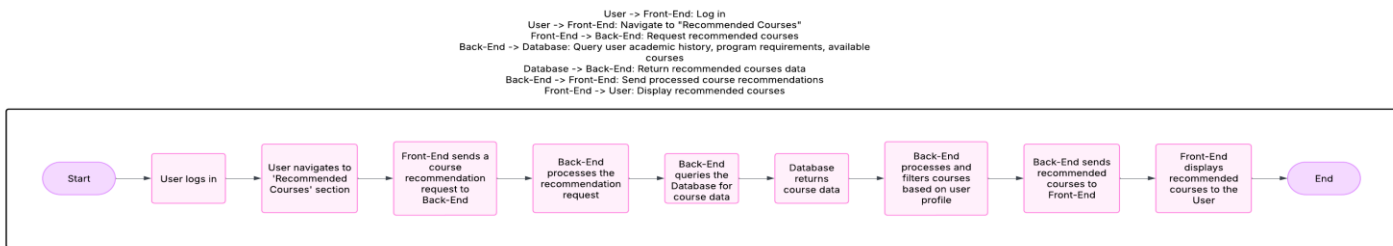
Type: *Performance*

Description: The app should be responsive and loading the result when the user clicks on a button should be fast. The application should be responsive and perform course analysis and provide recommendations in at least under 3 seconds and should take minimal time for the submission of the courses on the platform for students. Also, students should be able to expect a low waiting time for the login and registering to happen.

Use Cases

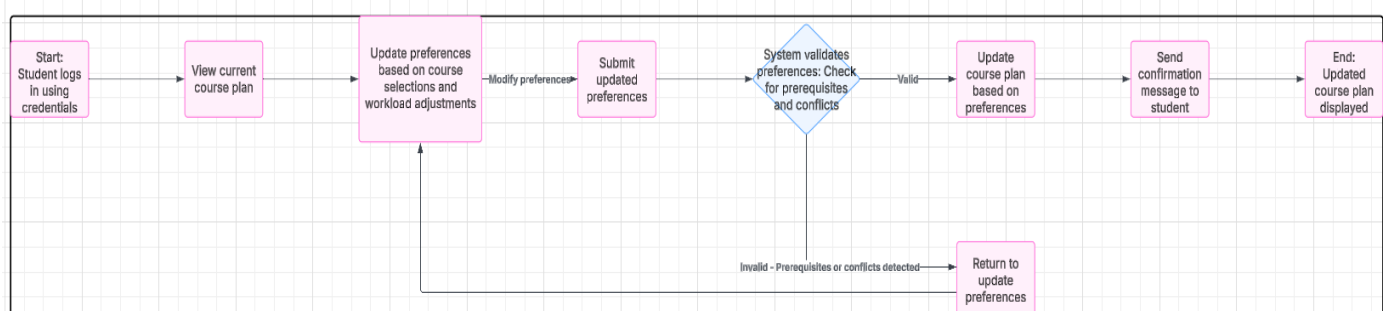
UC1: View Recommended Courses

Description: This use case allows a student to view recommended courses based on their academic progress, completed courses, and program requirements. (Evans Smith)



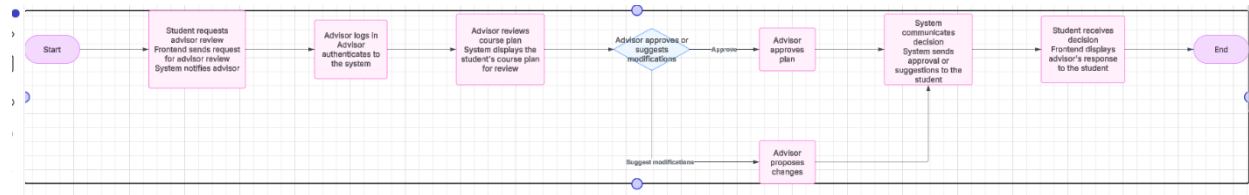
UC2: Updating course plan based on student preferences.

Description: This use case allows students to customize their generated course plan by selecting preferred courses or adjusting their workload.



UC3: Academic Advisor Review and Approval of Course Plan

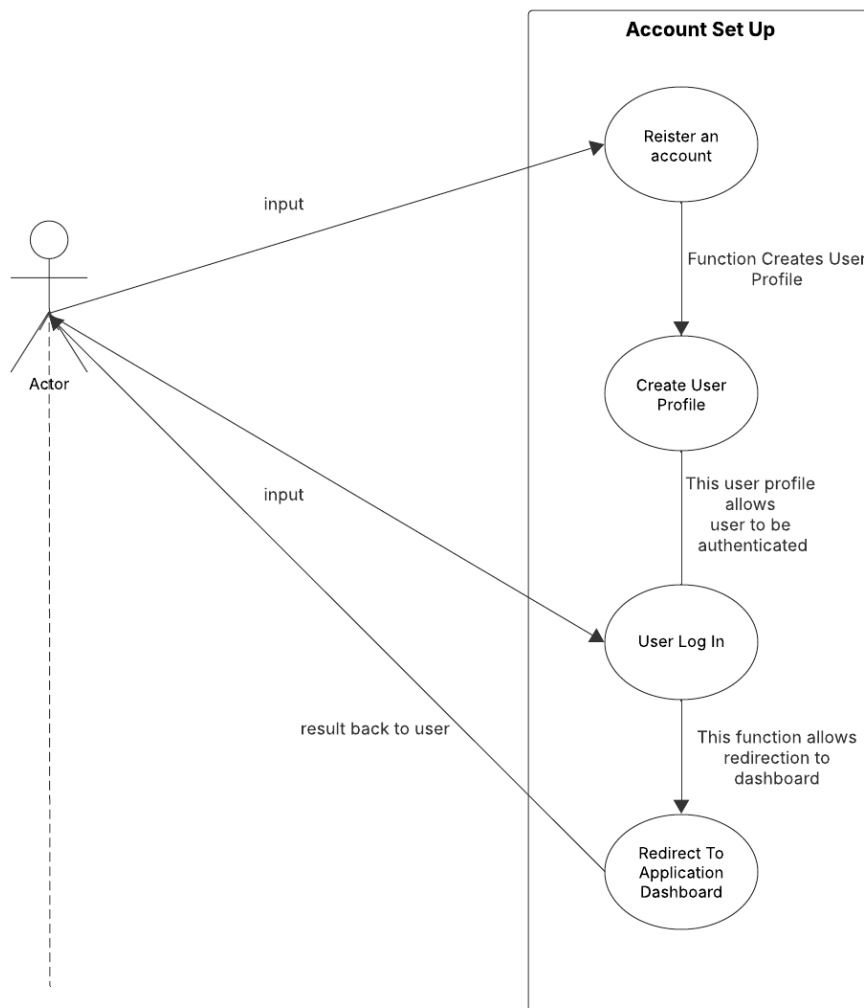
Description: This use case enables a student to request an academic advisor's review of the generated course plan. Once requested, the system notifies the advisor, who then logs in to review the plan. The advisor can approve the plan or suggest modifications, and the system communicates the decision back to the student.



Scenarios

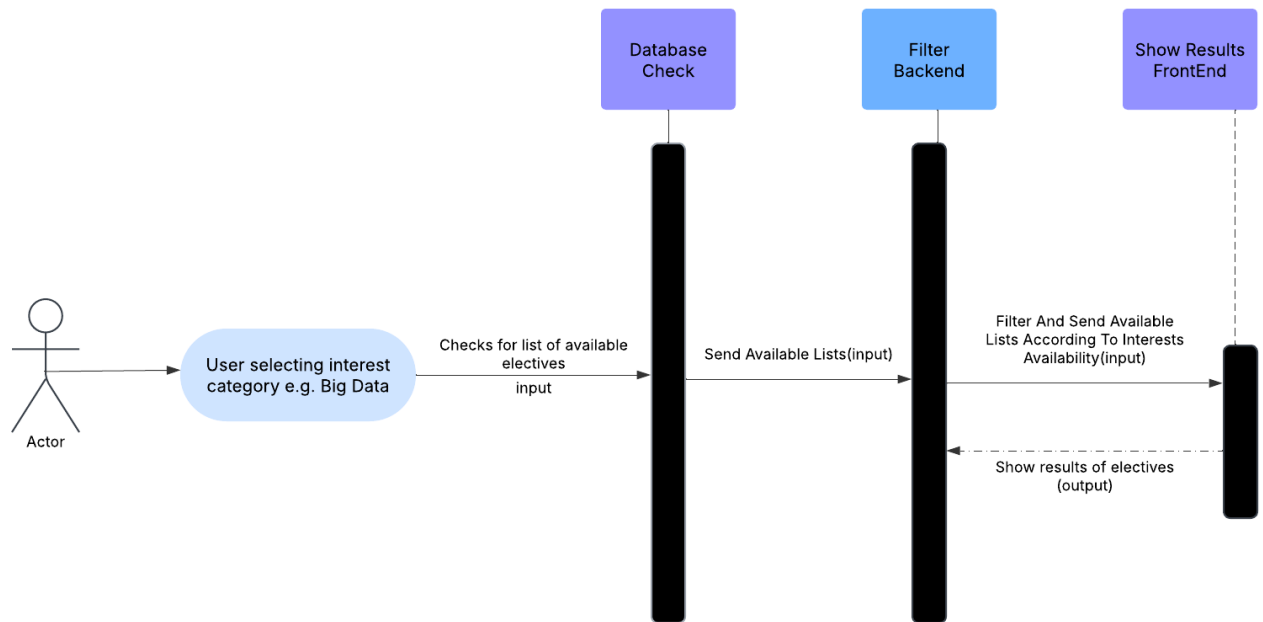
S1: User Account Set Up

Description: The student lands at the landing page to register their account and when they successfully register a user profile is created. In turn that user profile helps the student to log in and if the student successfully logs in then is redirected to the application dashboard where they can access other features of the application. So, the student gives two inputs at registration and login and gets result back after the redirection to application dashboard for the inputs.



S2: User Asks for Elective Suggestions Based on Interest

Description: A student goes to the course suggestions and enters “Big Data” as an interest while picking out course suggestions, so the application displays a recommendation of the related courses for the coming semester for selection.



S3: User Asks to Check for Graduation Timeline

Description: A student goes to graduation estimation timeline and enters all the courses that the student has finished. The system calculates the remaining required courses. The system asks for inputs like how many credits the student is going to take each term and determines the possible credits itself to provide an estimated graduation date.

